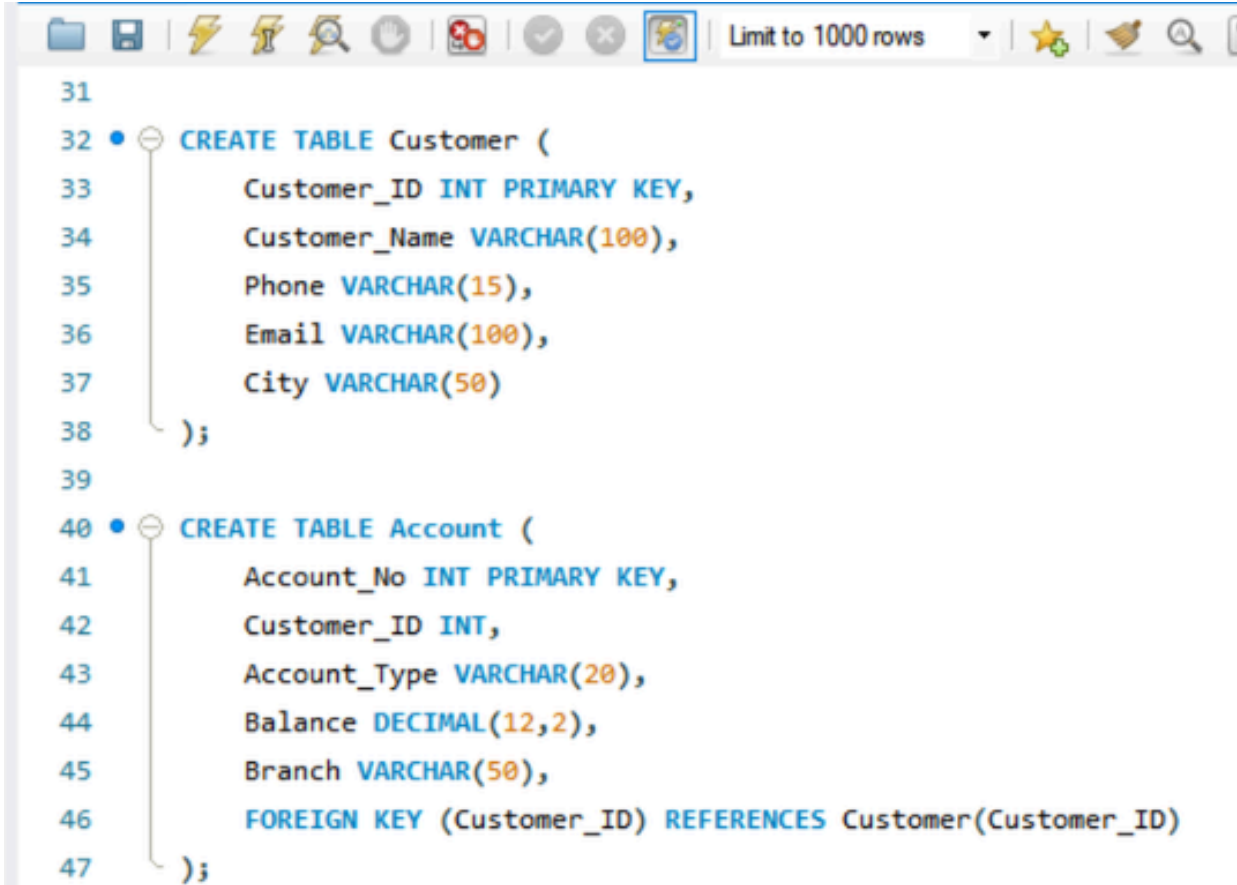


Week 3 Triggers and Procedures

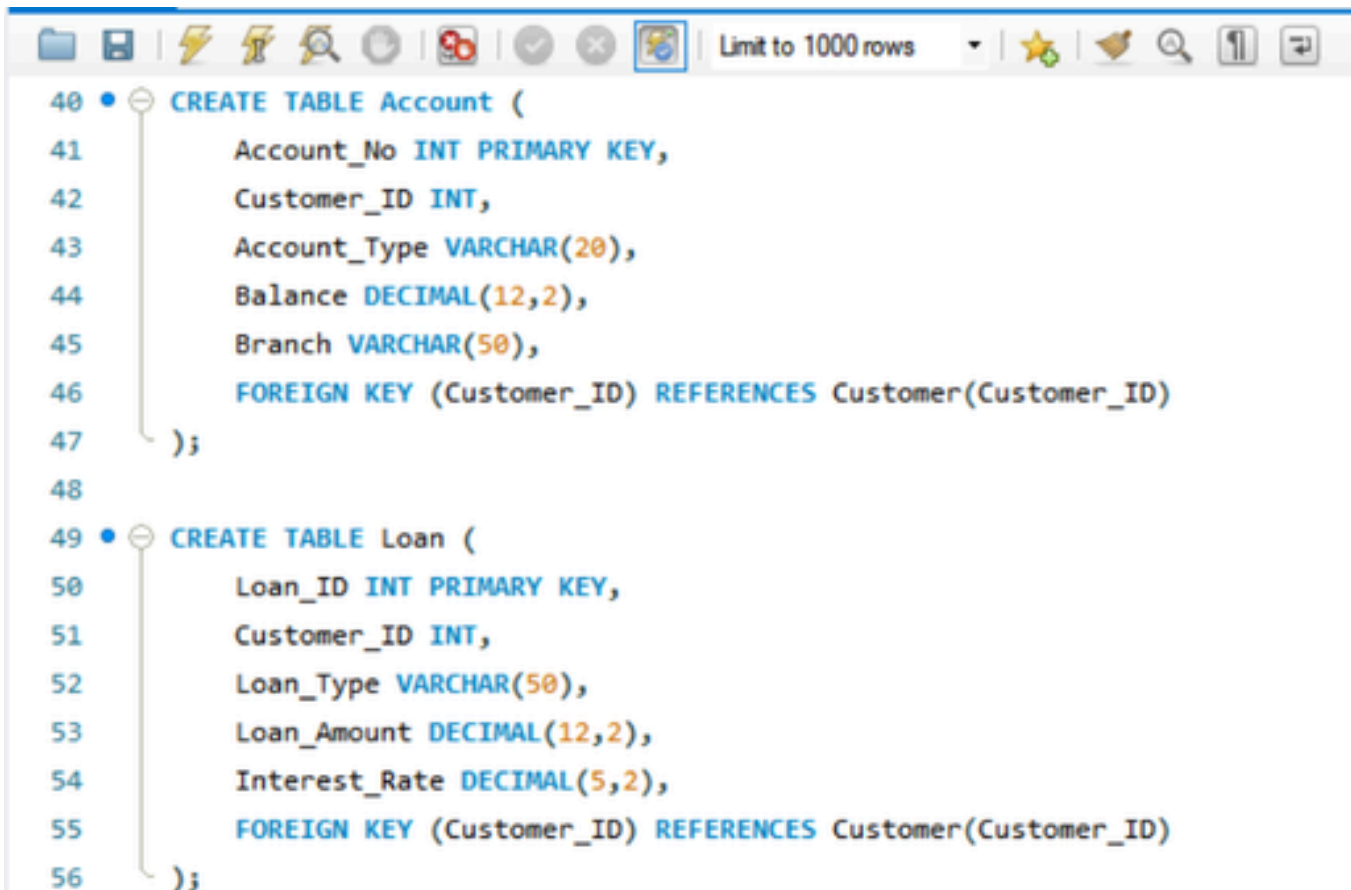
```
1  -- Setup Database
2  • CREATE DATABASE IF NOT EXISTS banking_db;
3  • USE banking_db;
4
5  -- Clean up existing objects to prevent duplicate errors
6  • DROP TRIGGER IF EXISTS CheckBalance;
7  • DROP TRIGGER IF EXISTS CheckTransactionAmount;
8  • DROP TRIGGER IF EXISTS Transaction_Audit;
9  • DROP TRIGGER IF EXISTS UpdateBalance_After_Transaction;
10 • DROP TRIGGER IF EXISTS CheckWithdrawalAmount;
11
12 • DROP PROCEDURE IF EXISTS GetAllCustomers;
13 • DROP PROCEDURE IF EXISTS GetAccountDetails;
14 • DROP PROCEDURE IF EXISTS GetCustomerAccounts;
15 • DROP PROCEDURE IF EXISTS DepositMoney;
16 • DROP PROCEDURE IF EXISTS WithdrawMoney;
17 • DROP PROCEDURE IF EXISTS TransferMoney;
18 • DROP PROCEDURE IF EXISTS GetCustomerLoans;
19 • DROP PROCEDURE IF EXISTS HighBalanceAccounts;
20 • DROP PROCEDURE IF EXISTS GetBalance;
```

```
19 • DROP PROCEDURE IF EXISTS HighBalanceAccounts;
20 • DROP PROCEDURE IF EXISTS GetBalance;
21
22 • DROP TABLE IF EXISTS Transaction_Audit;
23 • DROP TABLE IF EXISTS Bank_Transaction;
24 • DROP TABLE IF EXISTS Loan;
25 • DROP TABLE IF EXISTS Account;
26 • DROP TABLE IF EXISTS Customer;
27
28
29 -- TABLE CREATION & INITIAL SAMPLE DATA
30
31
32 • CREATE TABLE Customer (
33     Customer_ID INT PRIMARY KEY,
34     Customer_Name VARCHAR(100),
35     Phone VARCHAR(15),
36     Email VARCHAR(100),
37     City VARCHAR(50)
38 );
```



This screenshot shows a SQL IDE window with a toolbar at the top containing icons for file operations, execution, and search. The text area displays two SQL statements. The first statement, on line 32, creates a 'Customer' table with columns: 'Customer_ID' (INT PRIMARY KEY), 'Customer_Name' (VARCHAR(100)), 'Phone' (VARCHAR(15)), 'Email' (VARCHAR(100)), and 'City' (VARCHAR(50)). The second statement, on line 40, creates an 'Account' table with columns: 'Account_No' (INT PRIMARY KEY), 'Customer_ID' (INT), 'Account_Type' (VARCHAR(20)), 'Balance' (DECIMAL(12,2)), 'Branch' (VARCHAR(50)), and a foreign key constraint 'FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)'. The IDE interface includes a status bar at the top right indicating 'Limit to 1000 rows'.

```
31
32 • CREATE TABLE Customer (
33     Customer_ID INT PRIMARY KEY,
34     Customer_Name VARCHAR(100),
35     Phone VARCHAR(15),
36     Email VARCHAR(100),
37     City VARCHAR(50)
38 );
39
40 • CREATE TABLE Account (
41     Account_No INT PRIMARY KEY,
42     Customer_ID INT,
43     Account_Type VARCHAR(20),
44     Balance DECIMAL(12,2),
45     Branch VARCHAR(50),
46     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
47 );
```



This screenshot shows the same SQL IDE window, now displaying the completion of the 'Account' table and the creation of a 'Loan' table. The 'Account' table definition (lines 40-47) is identical to the previous block. The new statement, on line 49, creates a 'Loan' table with columns: 'Loan_ID' (INT PRIMARY KEY), 'Customer_ID' (INT), 'Loan_Type' (VARCHAR(50)), 'Loan_Amount' (DECIMAL(12,2)), 'Interest_Rate' (DECIMAL(5,2)), and a foreign key constraint 'FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)'. The IDE interface remains consistent with the previous screenshot, showing the same toolbar and status bar.

```
40 • CREATE TABLE Account (
41     Account_No INT PRIMARY KEY,
42     Customer_ID INT,
43     Account_Type VARCHAR(20),
44     Balance DECIMAL(12,2),
45     Branch VARCHAR(50),
46     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
47 );
48
49 • CREATE TABLE Loan (
50     Loan_ID INT PRIMARY KEY,
51     Customer_ID INT,
52     Loan_Type VARCHAR(50),
53     Loan_Amount DECIMAL(12,2),
54     Interest_Rate DECIMAL(5,2),
55     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
56 );
```



```
55     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID)
56 );
57
58 • ○ CREATE TABLE Bank_Transaction (
59     Transaction_ID INT PRIMARY KEY AUTO_INCREMENT,
60     Account_No INT,
61     Transaction_Type VARCHAR(20),
62     Amount DECIMAL(12,2),
63     FOREIGN KEY (Account_No) REFERENCES Account(Account_No)
64 );
65
66 • ○ CREATE TABLE Transaction_Audit (
67     Audit_ID INT PRIMARY KEY AUTO_INCREMENT,
68     Transaction_ID INT,
69     Account_No INT,
70     Transaction_Type VARCHAR(20),
71     Amount DECIMAL(12,2),
72     Audit_Date DATETIME DEFAULT CURRENT_TIMESTAMP
73 );
74
```



```
70     Transaction_Type VARCHAR(20),
71     Amount DECIMAL(12,2),
72     Audit_Date DATETIME DEFAULT CURRENT_TIMESTAMP
73 );
74
75 -- Insert Sample Data
76 • INSERT INTO Customer VALUES
77 (101, 'Ravi Kumar', '9876543210', 'ravi@gmail.com', 'Hyderabad'),
78 (102, 'Priya Sharma', '9876543211', 'priya@gmail.com', 'Vijayawada'),
79 (103, 'Arjun Reddy', '9876543212', 'arjun@gmail.com', 'Bangalore'),
80 (105, 'Kumar', '9876543214', 'kumar@gmail.com', 'Hyderabad'),
81 (107, 'Meena Reddy', '9876543216', 'meena@gmail.com', 'Mumbai'),
82 (110, 'Suresh Babu', '9876543219', 'suresh@gmail.com', 'Vijayawada');
83
84 • INSERT INTO Account VALUES
85 (10001, 101, 'Savings', 20000.00, 'Hyderabad'),
86 (10002, 102, 'Savings', 117000.00, 'Vijayawada'),
87 (10003, 103, 'Current', 130000.00, 'Bangalore'),
88 (10005, 105, 'Current', 90000.00, 'Hyderabad'),
89 (10007, 107, 'Current', 150000.00, 'Mumbai'),
90 (10010, 110, 'Current', 110000.00, 'Vijayawada');
```

```
91
92 • INSERT INTO Loan VALUES
93 (501, 101, 'Home Loan', 5000000.00, 7.50);
94
95
96 -- STORED PROCEDURES
97
98
99 -- 1. PROCEDURE TO DISPLAY ALL CUSTOMERS[cite: 2]
100 DELIMITER //
101
102 • CREATE PROCEDURE GetAllCustomers()
103 BEGIN
104     SELECT * FROM Customer;
105 END //
106
107 DELIMITER ;
108
109 • CALL GetAllCustomers();
110
```



```

109 • CALL GetAllCustomers();
110
111
112 -- 2. PROCEDURE TO DISPLAY ACCOUNT DETAILS FOR A GIVEN ACCOUNT NUMBER
113 DELIMITER //
114
115 • CREATE PROCEDURE GetAccountDetails(
116     IN p_Account_No INT
117 )
118 BEGIN
119     SELECT *
120     FROM Account
121     WHERE Account_No = p_Account_No;
122 END //
123
124 DELIMITER ;
125
126 • CALL GetAccountDetails(10001);
127
128
129 -- 3. PROCEDURE TO FIND CUSTOMER ACCOUNTS
130 DELIMITER //

```



```

130 DELIMITER //
131
132 • CREATE PROCEDURE GetCustomerAccounts(
133     IN p_Customer_ID INT
134 )
135 BEGIN
136     SELECT
137         C.Customer_ID,
138         C.Customer_Name,
139         A.Account_No,
140         A.Account_Type,
141         A.Balance,
142         A.Branch
143     FROM Customer C
144     JOIN Account A
145         ON C.Customer_ID = A.Customer_ID
146     WHERE C.Customer_ID = p_Customer_ID;
147 END //
148
149 DELIMITER ;
150
151 • CALL GetCustomerAccounts(101);

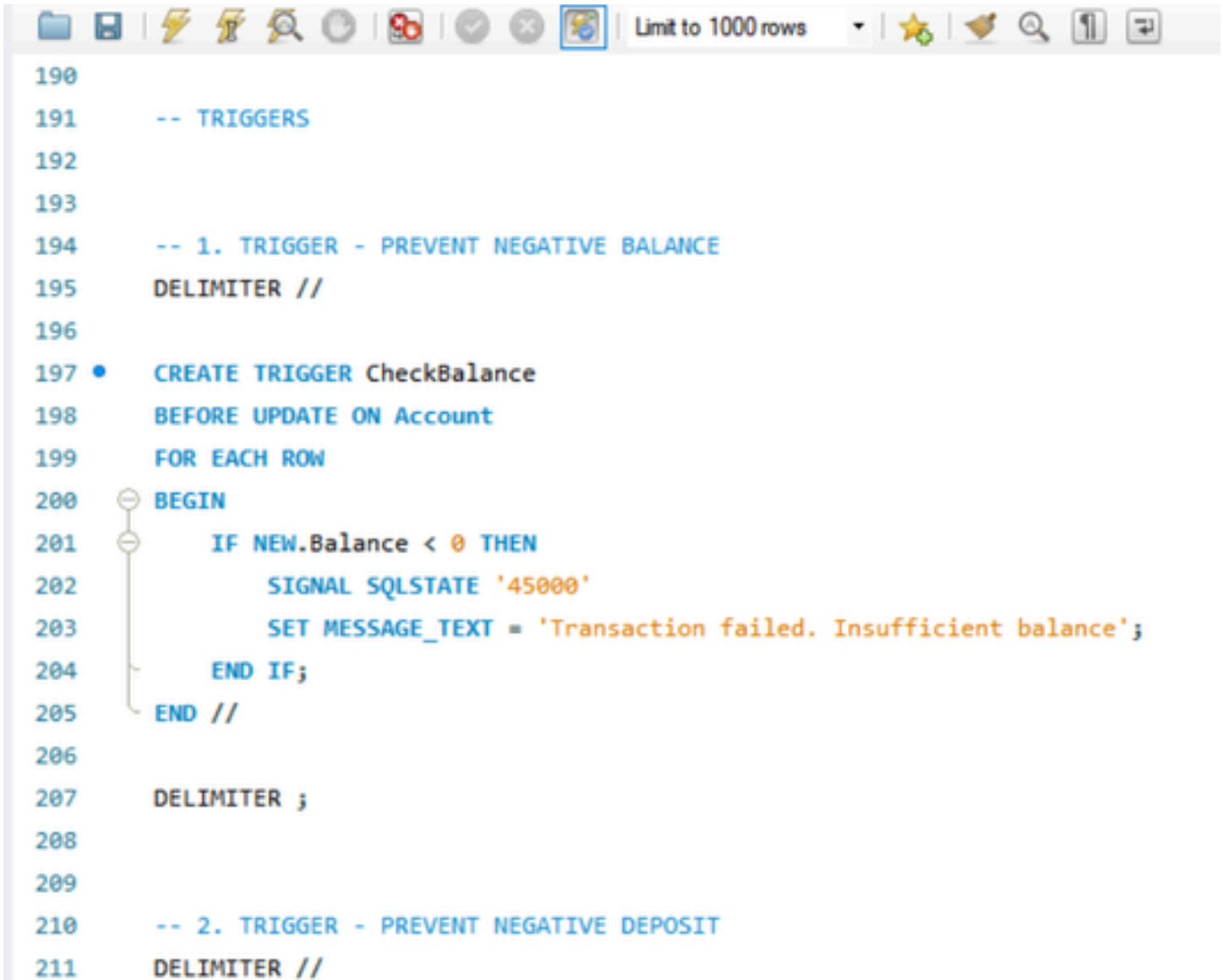
```



```
151 • CALL GetCustomerAccounts(101);
152
153
154 -- 4. PROCEDURE TO DEPOSIT MONEY
155 DELIMITER //
156
157 • CREATE PROCEDURE DepositMoney(
158     IN p_Account_No INT,
159     IN p_Amount DECIMAL(12,2)
160 )
161 BEGIN
162     UPDATE Account
163     SET Balance = Balance + p_Amount
164     WHERE Account_No = p_Account_No;
165 END //
166
167 DELIMITER ;
168
169 • CALL DepositMoney(10001, 5000);
```



```
169 • CALL DepositMoney(10001, 5000);
170
171
172 -- 5. PROCEDURE TO WITHDRAW MONEY
173 DELIMITER //
174
175 • CREATE PROCEDURE WithdrawMoney(
176     IN p_Account_No INT,
177     IN p_Amount DECIMAL(12,2)
178 )
179 BEGIN
180     UPDATE Account
181     SET Balance = Balance - p_Amount
182     WHERE Account_No = p_Account_No;
183 END //
184
185 DELIMITER ;
186
187 • CALL WithdrawMoney(10001, 3000);
188
189
190
```



```
190
191 -- TRIGGERS
192
193
194 -- 1. TRIGGER - PREVENT NEGATIVE BALANCE
195 DELIMITER //
196
197 • CREATE TRIGGER CheckBalance
198 BEFORE UPDATE ON Account
199 FOR EACH ROW
200 BEGIN
201     IF NEW.Balance < 0 THEN
202         SIGNAL SQLSTATE '45000'
203         SET MESSAGE_TEXT = 'Transaction failed. Insufficient balance';
204     END IF;
205 END //
206
207 DELIMITER ;
208
209
210 -- 2. TRIGGER - PREVENT NEGATIVE DEPOSIT
211 DELIMITER //
```

211 DELIMITER //

212

213 • CREATE TRIGGER CheckTransactionAmount

214 BEFORE INSERT ON Bank_Transaction

215 FOR EACH ROW

216 BEGIN

217 IF NEW.Amount <= 0 THEN

218 SIGNAL SQLSTATE '45000'

219 SET MESSAGE_TEXT = 'Transaction amount must be greater than zero';

220 END IF;

221 END //

222

223 DELIMITER ;

224

225

226 -- 3. TRIGGER - TRANSACTION AUDIT

227 DELIMITER //

226 -- 3. TRIGGER - TRANSACTION AUDIT

227 DELIMITER //

228

229 • CREATE TRIGGER Transaction_Audit

230 AFTER INSERT ON Bank_Transaction

231 FOR EACH ROW

232 BEGIN

233 INSERT INTO Transaction_Audit (

234 Transaction_ID,

235 Account_No,

236 Transaction_Type,

237 Amount

238)

239 VALUES (

240 NEW.Transaction_ID,

241 NEW.Account_No,

242 NEW.Transaction_Type,

243 NEW.Amount

244);

245 END //

246

247 DELIMITER ;



```

244     );
245 END //
246
247 DELIMITER ;
248
249 • INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount)
250 VALUES (10001, 'DEPOSIT', 2500);
251
252 • SELECT * FROM Transaction_Audit;
253
254
255 -- 4. TRIGGER - AUTOMATICALLY UPDATE ACCOUNT BALANCE
256 DELIMITER //

```



```

256 DELIMITER //
257
258 • CREATE TRIGGER UpdateBalance_After_Transaction
259 AFTER INSERT ON Bank_Transaction
260 FOR EACH ROW
261 BEGIN
262     IF NEW.Transaction_Type = 'DEPOSIT' THEN
263         UPDATE Account
264         SET Balance = Balance + NEW.Amount
265         WHERE Account_No = NEW.Account_No;
266     ELSEIF NEW.Transaction_Type = 'WITHDRAWAL' THEN
267         UPDATE Account
268         SET Balance = Balance - NEW.Amount
269         WHERE Account_No = NEW.Account_No;
270     END IF;
271 END //
272
273 DELIMITER ;
274
275 • INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount)
276 VALUES (10001, 'DEPOSIT', 5000);
277

```

```
274
275 • INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount)
276   VALUES (10001, 'DEPOSIT', 5000);
277
278 • SELECT * FROM Account WHERE Account_No = 10001;
279
280
281 -- 5. TRIGGER - PREVENT WITHDRAWAL ABOVE BALANCE
282 DELIMITER //
```

```
283
284 • CREATE TRIGGER CheckWithdrawalAmount
285   BEFORE INSERT ON Bank_Transaction
286   FOR EACH ROW
287   BEGIN
288     DECLARE CurrentBalance DECIMAL(12,2);
289
290     SELECT Balance INTO CurrentBalance
291     FROM Account
292     WHERE Account_No = NEW.Account_No;
293
294     IF NEW.Transaction_Type = 'WITHDRAWAL' AND NEW.Amount > CurrentBalance THEN
295       SIGNAL SQLSTATE '45000'
296       SET MESSAGE_TEXT = 'Withdrawal insufficient balance';
297     END IF;
298   END //
299
300 DELIMITER ;
```

```
304 -- ADVANCED STORED PROCEDURES
305 -- 1. PROCEDURE - ACCOUNT TRANSFER[cite: 2]
306 DELIMITER //
307
308 • CREATE PROCEDURE TransferMoney(
309     IN SenderAccount INT,
310     IN ReceiverAccount INT,
311     IN TransferAmount DECIMAL(12,2)
312 )
313 • BEGIN
314     UPDATE Account
315     SET Balance = Balance - TransferAmount
316     WHERE Account_No = SenderAccount;
317
318     UPDATE Account
319     SET Balance = Balance + TransferAmount
320     WHERE Account_No = ReceiverAccount;
321 END //
322
323 DELIMITER ;
```

```
325 • CALL TransferMoney(10001, 10002, 5000);
326
327 • SELECT * FROM Account
328     WHERE Account_No IN (10001, 10002);
329
330
331 -- 2. PROCEDURE - CUSTOMER LOAN DETAILS
332 DELIMITER //
```



```

334 • CREATE PROCEDURE GetCustomerLoans(
335     IN p_Customer_ID INT
336 )
337 BEGIN
338     SELECT
339         C.Customer_Name,
340         L.Loan_ID,
341         L.Loan_Type,
342         L.Loan_Amount,
343         L.Interest_Rate
344     FROM Customer C
345     JOIN Loan L
346         ON C.Customer_ID = L.Customer_ID
347     WHERE C.Customer_ID = p_Customer_ID;
348 END //
349
350 DELIMITER ;

```



```

358 • CREATE PROCEDURE HighBalanceAccounts(
359     IN MinimumBalance DECIMAL(12,2)
360 )
361 BEGIN
362     SELECT *
363     FROM Account
364     WHERE Balance > MinimumBalance
365     ORDER BY Balance DESC;
366 END //
367
368 DELIMITER ;
369
370 • CALL HighBalanceAccounts(50000);
371
372
373 -- 4. PROCEDURE WITH IN, OUT PARAMETERS
374 DELIMITER //

```

```
369
370 • CALL HighBalanceAccounts(50000);
371
372
373 -- 4. PROCEDURE WITH IN, OUT PARAMETERS
374 DELIMITER //
375
376 • CREATE PROCEDURE GetBalance(
377     IN p_Account_No INT,
378     OUT p_Balance DECIMAL(12,2)
379 )
380 • BEGIN
381     SELECT Balance
382     INTO p_Balance
383     FROM Account
384     WHERE Account_No = p_Account_No;
385 END //
386
387 DELIMITER ;
388
389 • CALL GetBalance(10001, @CurrentBalance);
390 • SELECT @CurrentBalance;
```

OUTPUTS

Result Grid  Filter Rows: | Export:  | Wrap Cell Content: 

	Customer_ID	Customer_Name	Phone	Email	City
▶	101	Ravi Kumar	9876543210	ravi@gmail.com	Hyderabad
	102	Priya Sharma	9876543211	priya@gmail.com	Vijayawada
	103	Arjun Reddy	9876543212	arjun@gmail.com	Bangalore
	105	Kumar	9876543214	kumar@gmail.com	Hyderabad
	107	Meena Reddy	9876543216	meena@gmail.com	Mumbai
	110	Suresh Babu	9876543219	suresh@gmail.com	Vijayawada

Result Grid  Filter Rows: | Export:  | Wrap Cell Content: 

	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	20000.00	Hyderabad

Result Grid  Filter Rows: | Export:  | Wrap Cell Content: 

	Customer_ID	Customer_Name	Account_No	Account_Type	Balance	Branch
▶	101	Ravi Kumar	10001	Savings	20000.00	Hyderabad

Result Grid   Filter Rows: | Edit:    | Export/Import:   | Wrap Cell Content: 

	Audit_ID	Transaction_ID	Account_No	Transaction_Type	Amount	Audit_Date
▶	1	1	10001	DEPOSIT	2500.00	2026-08-28 12:13:57
•	NULL	NULL	NULL	NULL	NULL	NULL

Result Grid   Filter Rows: | Edit:    | Export/

	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	27000.00	Hyderabad
•	NULL	NULL	NULL	NULL	NULL

Result Grid		Filter Rows:	Edit: 		Exp
	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10001	101	Savings	22000.00	Hyderabad
	10002	102	Savings	122000.00	Vijayawada
✱	NULL	NULL	NULL	NULL	NULL

Result Grid


Filter Rows:

Export:


Wrap Cell Content:


	Customer_Name	Loan_ID	Loan_Type	Loan_Amount	Interest_Rate
▶	Ravi Kumar	501	Home Loan	5000000.00	7.50

Result Grid


Filter Rows:

Export:


Wrap Cell Content:


	Account_No	Customer_ID	Account_Type	Balance	Branch
▶	10007	107	Current	150000.00	Mumbai
	10003	103	Current	130000.00	Bangalore
	10002	102	Savings	122000.00	Vijayawada
	10010	110	Current	110000.00	Vijayawada
	10005	105	Current	90000.00	Hyderabad

Result Grid			Filter Rows:
	@CurrentBalance		
▶	22000.00		

Output



Action Output



	#	Time	Action
	1	12:13:56	CREATE DATABASE IF NOT EXISTS banking_db
	2	12:13:56	USE banking_db
	3	12:13:56	DROP TRIGGER IF EXISTS CheckBalance
	4	12:13:56	DROP TRIGGER IF EXISTS CheckTransactionAmount
	5	12:13:56	DROP TRIGGER IF EXISTS Transaction_Audit
	6	12:13:56	DROP TRIGGER IF EXISTS UpdateBalance_After_Transaction
	7	12:13:56	DROP TRIGGER IF EXISTS CheckWithdrawalAmount
	8	12:13:56	DROP PROCEDURE IF EXISTS GetAllCustomers
	9	12:13:56	DROP PROCEDURE IF EXISTS GetAccountDetails
	10	12:13:56	DROP PROCEDURE IF EXISTS GetCustomerAccounts
	11	12:13:56	DROP PROCEDURE IF EXISTS DepositMoney

	11	12:13:56	DROP PROCEDURE IF EXISTS DepositMoney
	12	12:13:56	DROP PROCEDURE IF EXISTS WithdrawMoney
	13	12:13:56	DROP PROCEDURE IF EXISTS TransferMoney
	14	12:13:56	DROP PROCEDURE IF EXISTS GetCustomerLoans
	15	12:13:56	DROP PROCEDURE IF EXISTS HighBalanceAccounts
	16	12:13:56	DROP PROCEDURE IF EXISTS GetBalance
	17	12:13:56	DROP TABLE IF EXISTS Transaction_Audit
	18	12:13:56	DROP TABLE IF EXISTS Bank_Transaction
	19	12:13:56	DROP TABLE IF EXISTS Loan
	20	12:13:56	DROP TABLE IF EXISTS Account
	21	12:13:56	DROP TABLE IF EXISTS Customer

Output		
Action Output		
#	Time	Action
✓ 20	12:13:56	DROP TABLE IF EXISTS Account
✓ 21	12:13:56	DROP TABLE IF EXISTS Customer
✓ 22	12:13:56	CREATE TABLE Customer (Customer_ID INT PRIMARY KEY, Customer_Name VARCHAR(100), Phone VARCHAR(15),
✓ 23	12:13:56	CREATE TABLE Account (Account_No INT PRIMARY KEY, Customer_ID INT, Account_Type VARCHAR(20), Balance
✓ 24	12:13:56	CREATE TABLE Loan (Loan_ID INT PRIMARY KEY, Customer_ID INT, Loan_Type VARCHAR(50), Loan_Amount DEC
✓ 25	12:13:56	CREATE TABLE Bank_Transaction (Transaction_ID INT PRIMARY KEY AUTO_INCREMENT, Account_No INT, Transact
✓ 26	12:13:57	CREATE TABLE Transaction_Audit (Audit_ID INT PRIMARY KEY AUTO_INCREMENT, Transaction_ID INT, Account_No
✓ 27	12:13:57	INSERT INTO Customer VALUES (101, 'Ravi Kumar', '9876543210', 'ravi@gmail.com', 'Hyderabad'), (102, 'Priya Shama', '9876543
✓ 28	12:13:57	INSERT INTO Account VALUES (10001, 101, 'Savings', 20000.00, 'Hyderabad'), (10002, 102, 'Savings', 117000.00, 'Vijayawada'),
✓ 29	12:13:57	INSERT INTO Loan VALUES (501, 101, 'Home Loan', 5000000.00, 7.50)
✓ 30	12:13:57	CREATE PROCEDURE GetAllCustomers() BEGIN SELECT * FROM Customer; END

Output		
Action Output		
#	Time	Action
✓ 30	12:13:57	CREATE PROCEDURE GetAllCustomers() BEGIN SELECT * FROM Customer; END
✓ 31	12:13:57	CALL GetAllCustomers()
✓ 32	12:13:57	CREATE PROCEDURE GetAccountDetails(IN p_Account_No INT) BEGIN SELECT * FROM Account WHERE Acco
✓ 33	12:13:57	CALL GetAccountDetails(10001)
✓ 34	12:13:57	CREATE PROCEDURE GetCustomerAccounts(IN p_Customer_ID INT) BEGIN SELECT C.Customer_ID, C.Custo
✓ 35	12:13:57	CALL GetCustomerAccounts(101)
✓ 36	12:13:57	CREATE PROCEDURE DepositMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN UPDATE Account
✓ 37	12:13:57	CALL DepositMoney(10001, 5000)
✓ 38	12:13:57	CREATE PROCEDURE WithdrawMoney(IN p_Account_No INT, IN p_Amount DECIMAL(12,2)) BEGIN UPDATE Accour
✓ 39	12:13:57	CALL WithdrawMoney(10001, 3000)
✓ 40	12:13:57	CREATE TRIGGER CheckBalance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.Balance < 0 THEN

Output		
Action Output		
#	Time	Action
✓ 41	12:13:57	CREATE TRIGGER CheckTransactionAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW BEGIN IF NEW.Am
✓ 42	12:13:57	CREATE TRIGGER Transaction_Audit AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN INSERT INTO Transa
✓ 43	12:13:57	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', 2500)
✓ 44	12:13:57	SELECT * FROM Transaction_Audit LIMIT 0, 1000
✓ 45	12:13:57	CREATE TRIGGER UpdateBalance_After_Transaction AFTER INSERT ON Bank_Transaction FOR EACH ROW BEGIN IF NE
✓ 46	12:13:57	INSERT INTO Bank_Transaction (Account_No, Transaction_Type, Amount) VALUES (10001, 'DEPOSIT', 5000)
✓ 47	12:13:57	SELECT * FROM Account WHERE Account_No = 10001 LIMIT 0, 1000
✓ 48	12:13:57	CREATE TRIGGER CheckWithdrawalAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW BEGIN DECLARE C
✓ 49	12:13:57	CREATE PROCEDURE TransferMoney(IN SenderAccount INT, IN ReceiverAccount INT, IN TransferAmount DECIMAL(1
✓ 50	12:13:57	CALL TransferMoney(10001, 10002, 5000)
✓ 51	12:13:57	SELECT * FROM Account WHERE Account_No IN (10001, 10002) LIMIT 0, 1000

Output		
Action Output		
#	Time	Action
✓ 48	12:13:57	CREATE TRIGGER CheckWithdrawalAmount BEFORE INSERT ON Bank_Transaction FOR EACH ROW BEGIN DECLARE Cu
✓ 49	12:13:57	CREATE PROCEDURE TransferMoney(IN SenderAccount INT, IN ReceiverAccount INT, IN TransferAmount DECIMAL(1
✓ 50	12:13:57	CALL TransferMoney(10001, 10002, 5000)
✓ 51	12:13:57	SELECT * FROM Account WHERE Account_No IN (10001, 10002) LIMIT 0, 1000
✓ 52	12:13:57	CREATE PROCEDURE GetCustomerLoans(IN p_Customer_ID INT) BEGIN SELECT C.Customer_Name, L.Loan_I
✓ 53	12:13:57	CALL GetCustomerLoans(101)
✓ 54	12:13:57	CREATE PROCEDURE HighBalanceAccounts(IN MinimumBalance DECIMAL(12,2)) BEGIN SELECT * FROM Account
✓ 55	12:13:57	CALL HighBalanceAccounts(50000)
✓ 56	12:13:57	CREATE PROCEDURE GetBalance(IN p_Account_No INT, OUT p_Balance DECIMAL(12,2)) BEGIN SELECT Balance
✓ 57	12:13:57	CALL GetBalance(10001, @CurrentBalance)
✓ 58	12:13:57	SELECT @CurrentBalance LIMIT 0, 1000